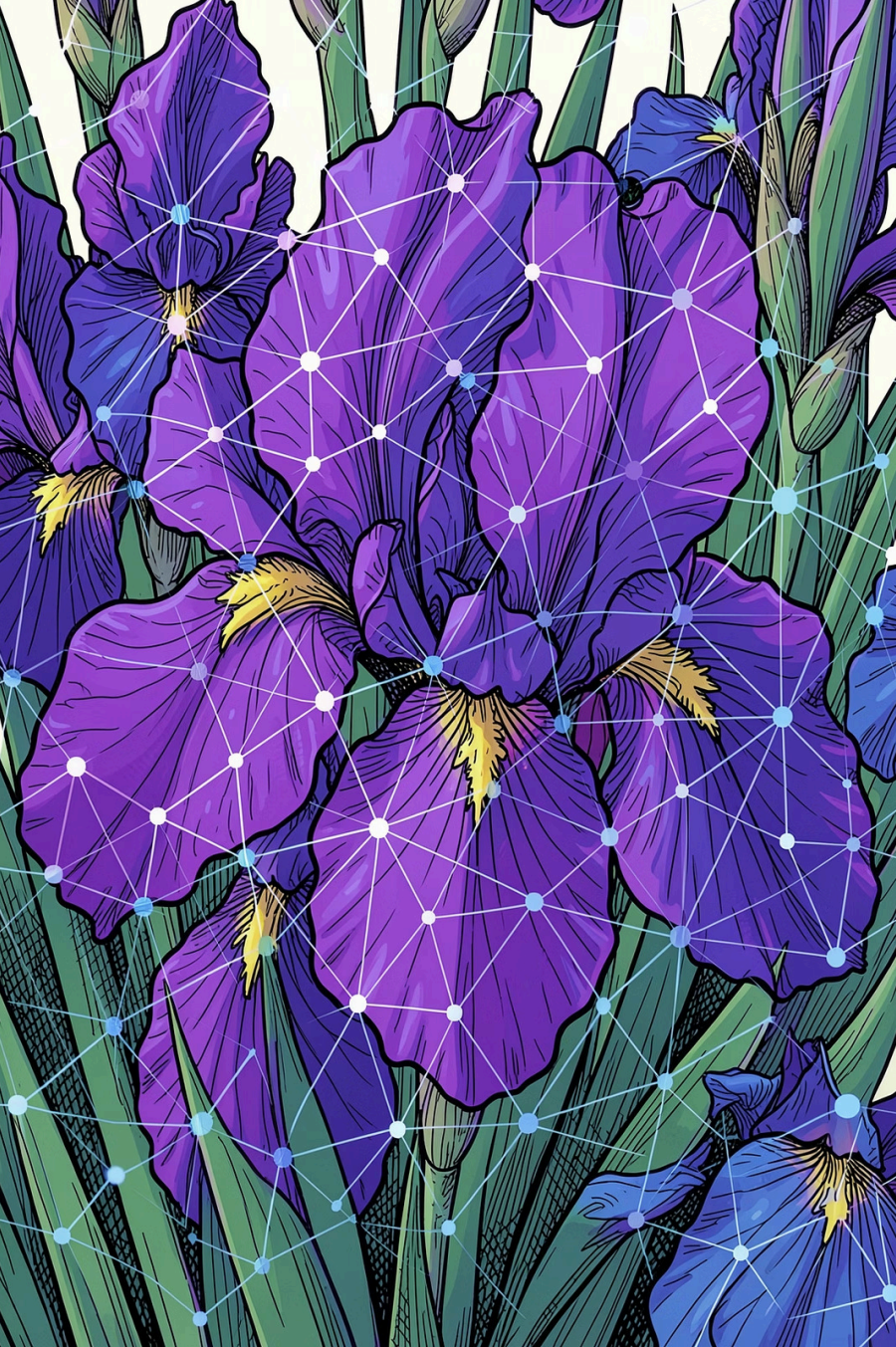




Deep Learning — Iris

Un percorso completo per costruire e testare una rete neurale sul dataset Iris di Kaggle: dall'analisi esplorativa con Power BI alla classificazione con Keras.

Indice

01

Workflow del Progetto

Panoramica dell'intero flusso di lavoro

02

EDA con Power BI

Analisi esplorativa e visualizzazione dei dati

03

Preprocessing

Label encoding, split e standard scaling

04

Reti Neurali - Keras

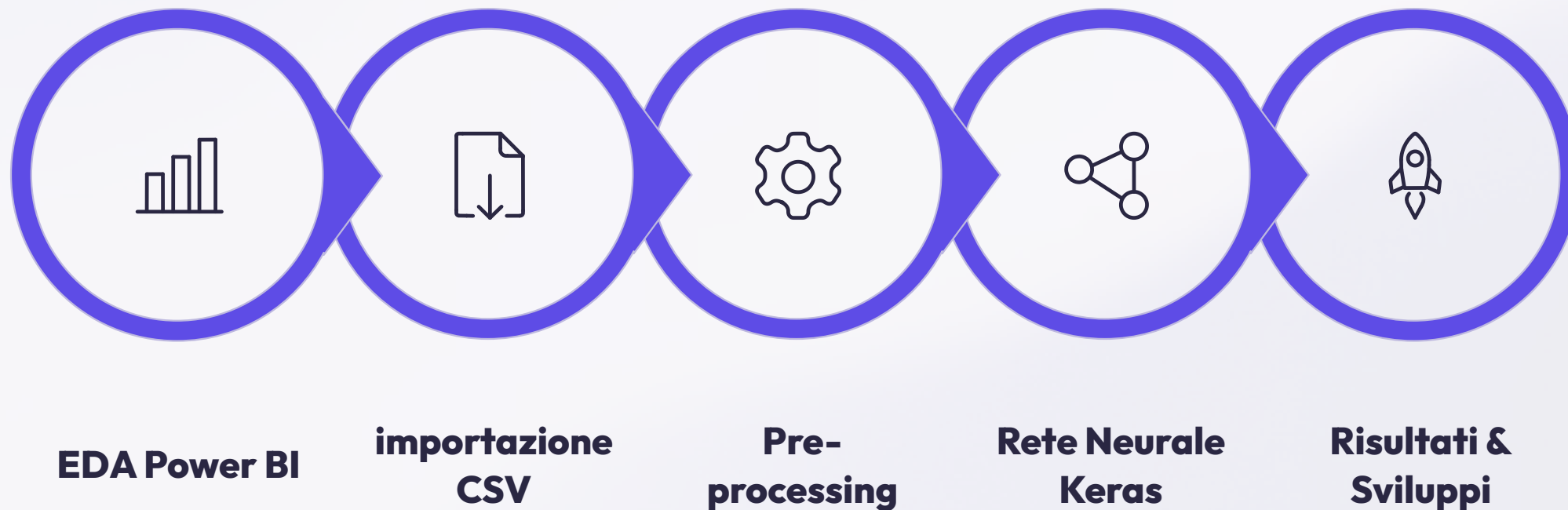
Fondamenti teorici e ottimizzatori

05

Applicazione, Risultati e Sviluppi Futuri

Sperimentazione, metriche e roadmap

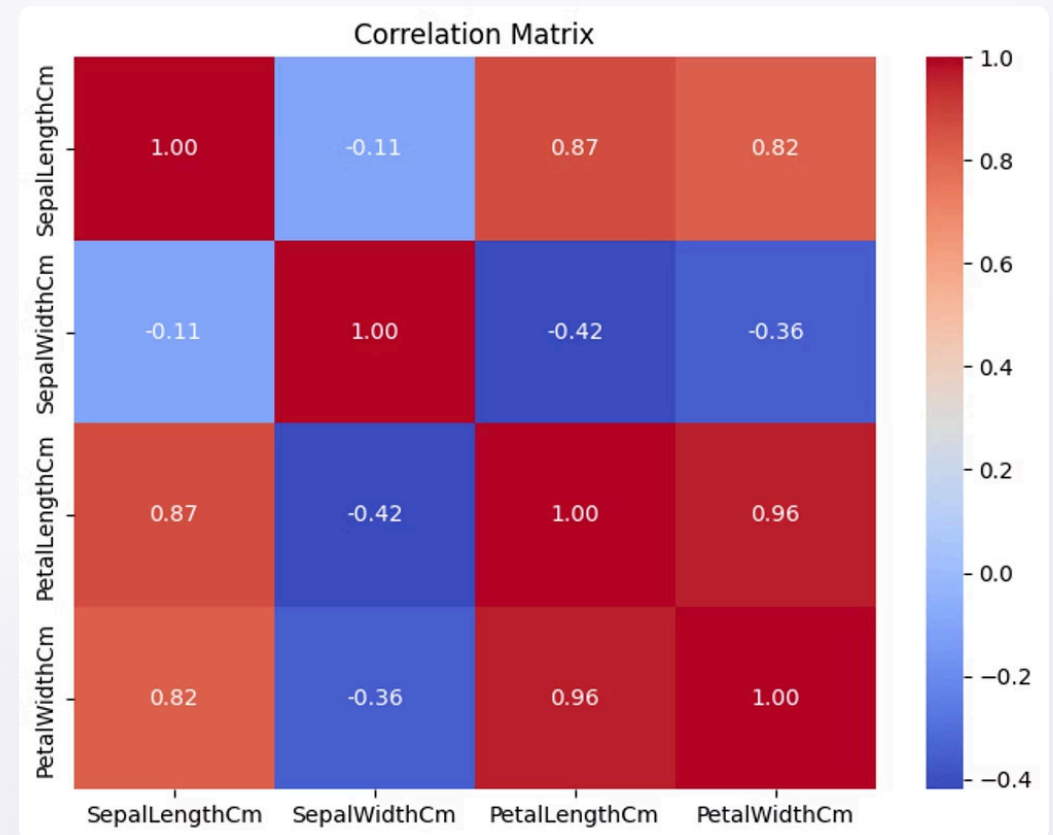
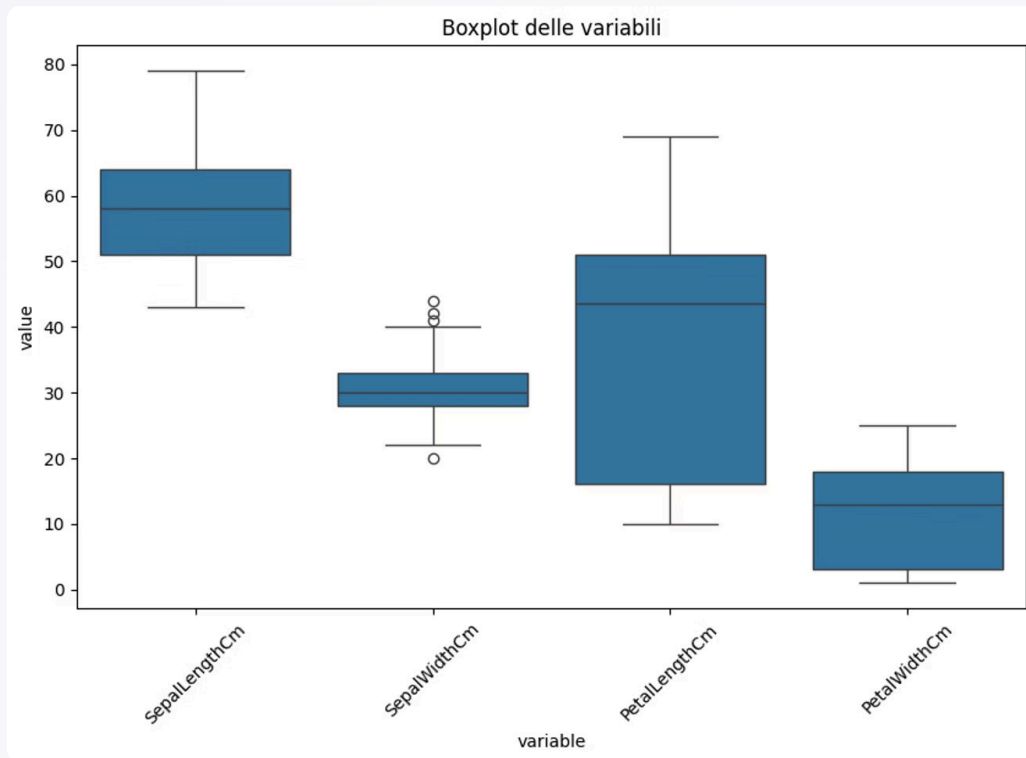
Workflow del Progetto



Ogni fase prepara i dati e il modello per il passo successivo, garantendo un flusso strutturato e riproducibile.

Analisi Esplorativa con Power BI

Power BI consente di esplorare il dataset prima di scrivere codice, identificando distribuzioni, correlazioni e anomalie in modo intuitivo.

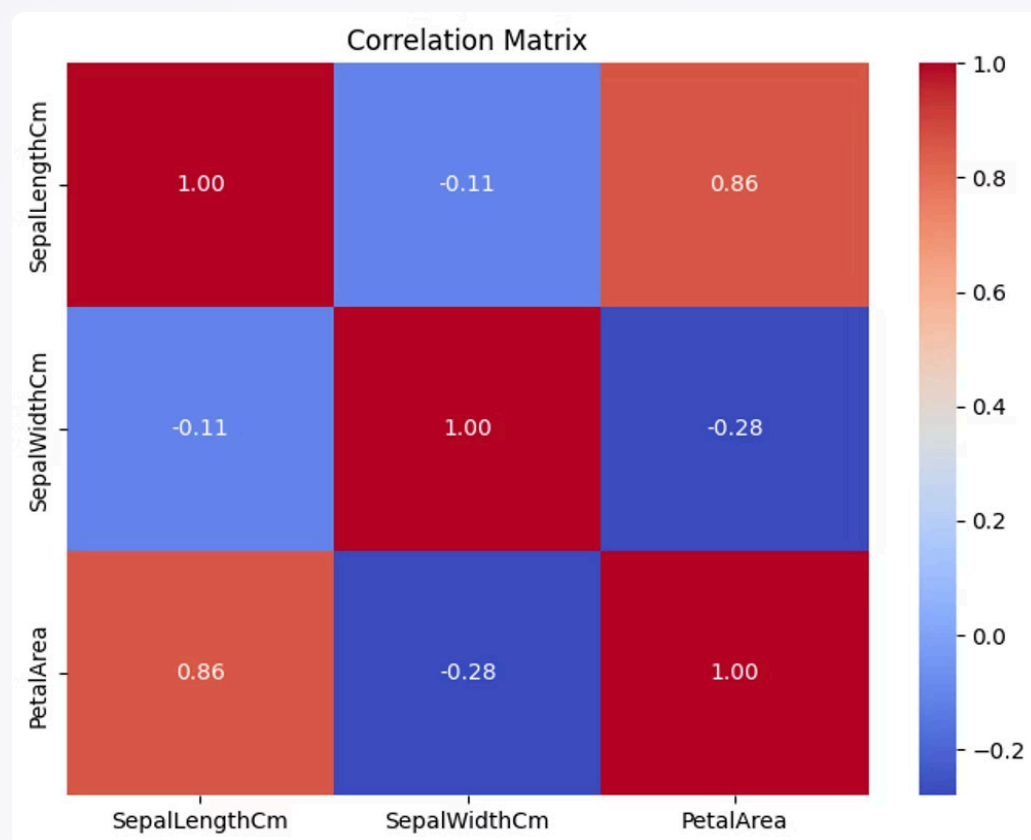


Nel boxplot si notano alcuni punti più distanti rispetto alla distribuzione centrale, indicando possibili outliers.

La correlation matrix aiuta a diagnosticare la multicollinearità.

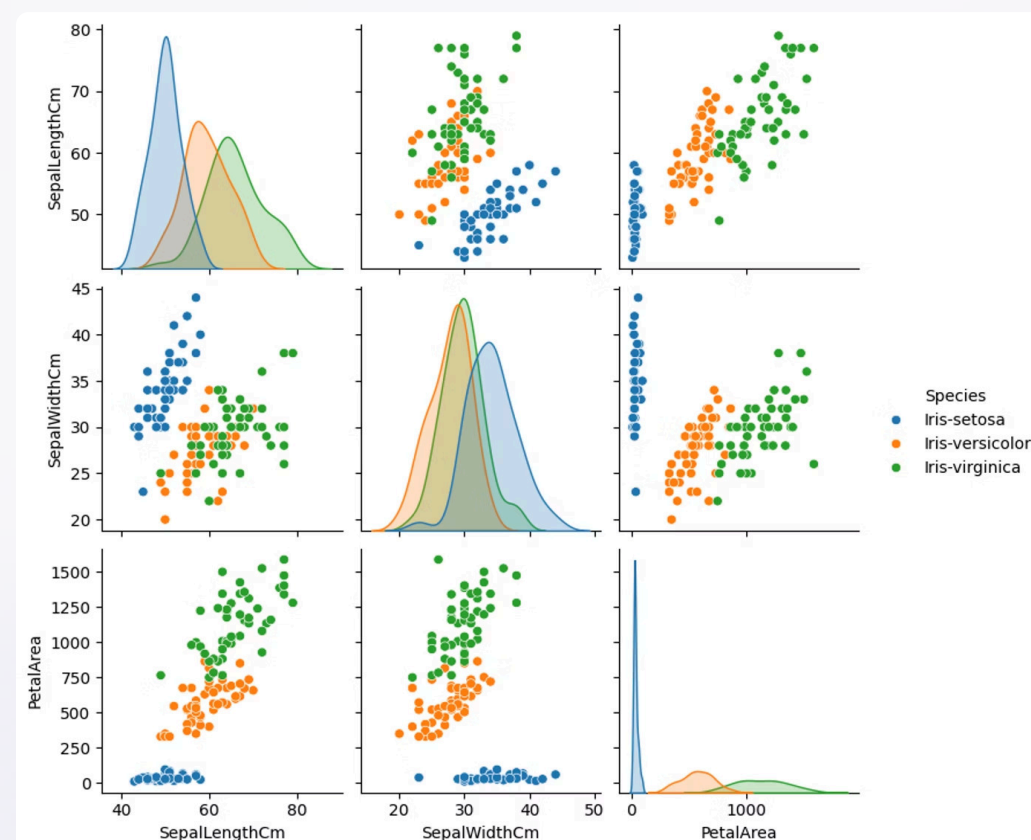
Analisi Esplorativa con Power BI – Feature Engineering

La feature engineered dell'*area del petalo* aiuta a rendere più leggibili le relazioni nel dataset e a separare meglio le classi.



Correlation Matrix dopo feature engineering

La nuova feature migliora la struttura delle correlazioni e rende più evidenti i legami più utili tra le variabili.



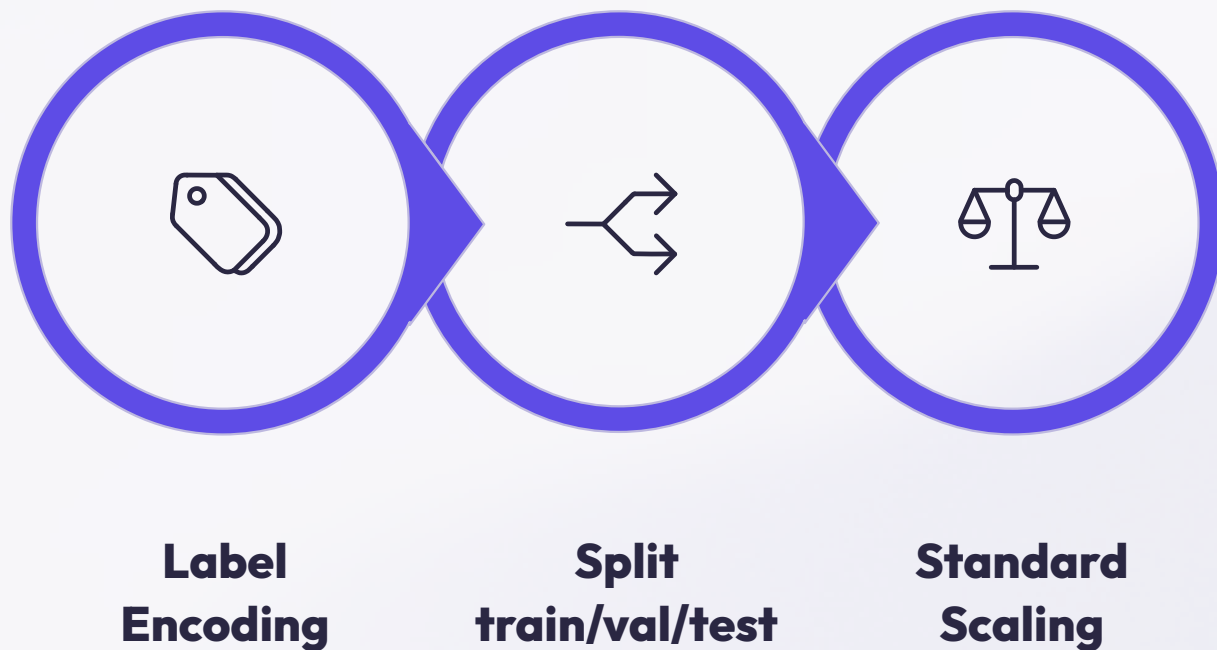
Scatter plot + densità

Lo scatter plot mostra una separabilità molto chiara tra Setosa, Versicolor e Virginica.

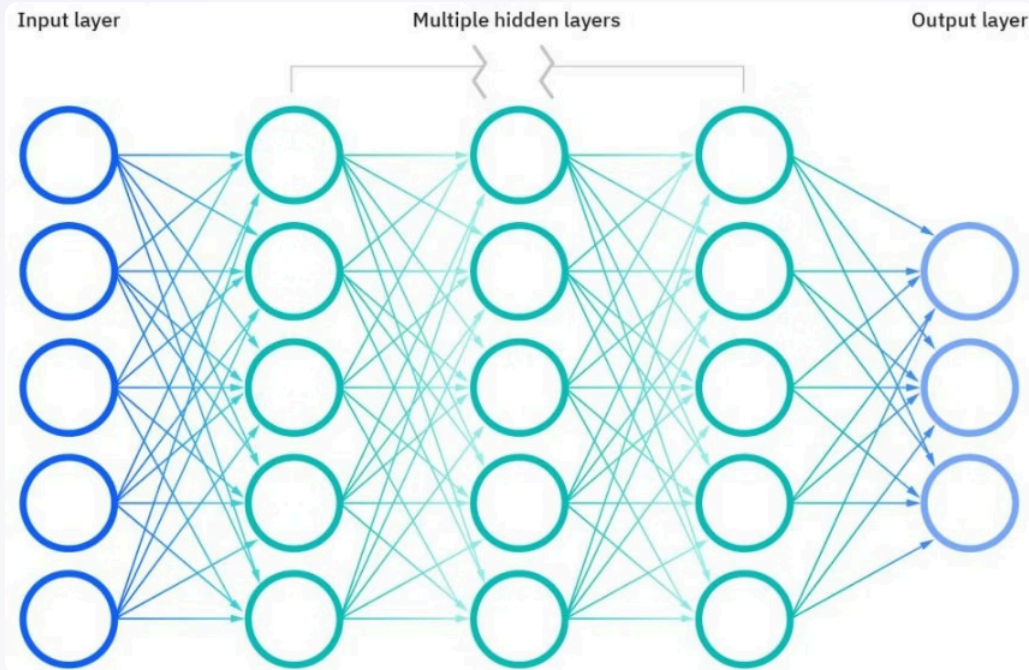
Le densità rafforzano la distinzione visiva delle tre classi nelle proiezioni.

Preprocessing

Prima di alimentare la rete neurale, i dati vengono preparati in tre passaggi fondamentali.



Lo **Standard Scaling** è particolarmente critico per le reti neurali: feature su scale diverse causano convergenza lenta o instabile durante l'addestramento.



TEORIA

Multilayer Perceptron (MLP)

La famiglia di modelli che abbiamo testato utilizzando la libreria Keras.

Un MLP è una rete neurale feedforward composta da strati di neuroni interconnessi, ideale per cogliere relazioni non lineari nei problemi di classificazione come il dataset Iris.

Gli Ottimizzatori

Gli **optimizers** aggiornano i pesi della rete per minimizzare la funzione di perdita. Scegliere quello giusto influenza velocità di convergenza e qualità del modello. Alcuni esempi di optimizers sono:



SGD

Aggiornamento basato sul gradiente semplice. Robusto ma può convergere lentamente.



Adam

Adatta il learning rate per ogni parametro. Tra i più usati nel deep learning moderno.



RMSprop

Adatta il learning rate in base alla media mobile dei gradienti recenti. Efficace su problemi non stazionari.

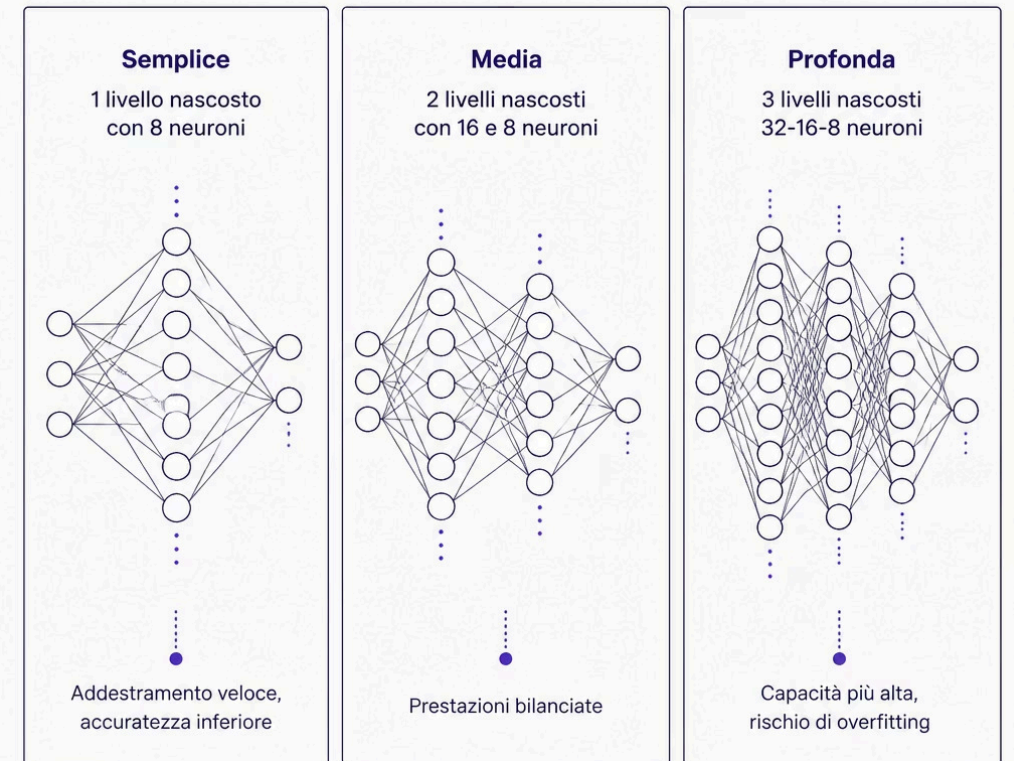
Applicazione della Rete Neurale

Architettura Sperimentata

- Rete media: 2 hidden layers (16 + 8 neuroni)

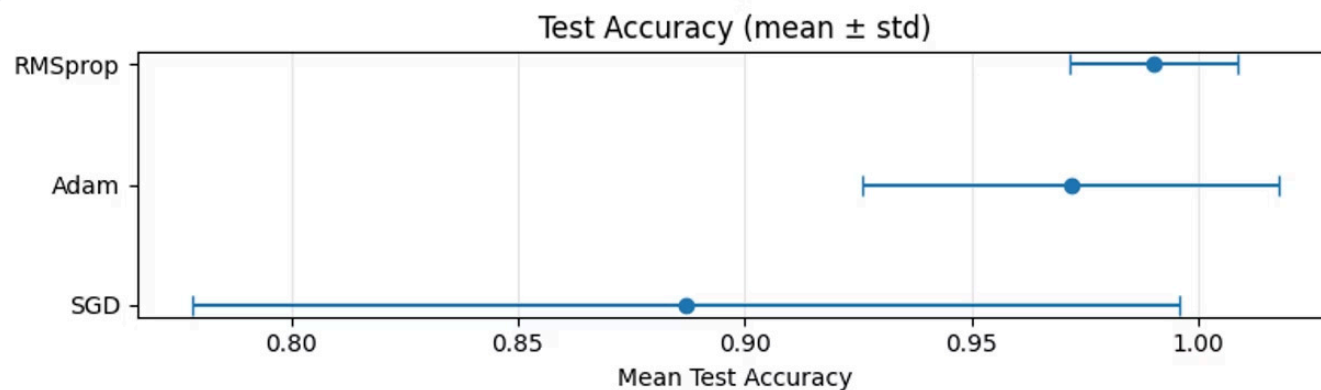
Configurazione del Modello

- Attivazione: **ReLU** (hidden), **Softmax** (output)
- Loss: **categorical_crossentropy**
- Optimizer: Adam, SGD, **RMSprop** (confronto)
- Epoche: 50–80-100 con early stopping



Risultati Ottenuti

Valutazione tramite accuratezza, loss e matrice di confusione su train, validation e test set per confrontare architetture e ottimizzatori.



📄 Conclusioni dai Risultati

- **RMSprop** raggiunge la migliore accuratezza sul test set, con valori intorno al **97-98%**.
- **Adam** mostra buone prestazioni, ma con una variabilità maggiore tra le esecuzioni.
- **SGD** presenta prestazioni inferiori e meno stabili rispetto agli altri optimizer.
- **Conclusione:** **RMSprop** è l'optimizer più affidabile per questo dataset.

Sviluppi Futuri

Hyperparameter Tuning

Grid search o random search per ottimizzare learning rate, numero di neuroni e regolarizzazione (dropout, L2).

Sperimentazione Avanzata

Estendere l'analisi ad altri dataset, esplorare tecniche di cross-validation e confrontare con modelli classici (Random Forest, SVM).

Testing Semplificato

Possibilità di passare direttamente tramite endpoint API l'ottimizzatore scelto e il numero di epochs

Testing su Reti Neurali di altre Dimensioni

L'unica dimensione di rete testata nel nostro progetto è stata quella con 2 hidden layer (rispettivamente con 16 e 8 neuroni). Estendere la possibilità di testare su reti più piccole (1 layer nascosto con 8 neuroni) o più grandi (3 layer nascosti 32-16-8)